



NU MOA



React JSX and Components

FOUNDED FOR FILIPINOS, BUILT FOR THE NATION

At the end of the lesson(s), students must be able to:

- Understand what JSX and its role in React Development
- Learn how to use JSX syntax to create React Elements
- Understand how to use JSX Attributes to customize React Components
- Know how to create reusable styled components in React
- Learn how to use React routing for web navigation.

What is JSX?

JSX stands for **JavaScript XML**. It is a syntax extension for JavaScript that allows developers to write HTML-like code directly inside JavaScript files.

Although JSX looks similar to HTML, it is not HTML. It is a special syntax that React converts into standard JavaScript before the browser executes it. This conversion is performed by a compiler such as **Babel**.

JSX makes React applications easier to read, write, and maintain because developers can combine the user interface and application logic within the same file.

Key Point: JSX is not a programming language. It is a syntax extension that simplifies the creation of React elements.

Why Do We Use JSX?



NU MOA



Before JSX was introduced, developers had to use the `React.createElement()` function to create user interface elements. While functional, this approach becomes difficult to read and maintain as applications grow larger.

Without JSX

```
const element = React.createElement(  
  "h1",  
  null,  
  "Welcome to ReactJS"  
);
```

With JSX

```
const element = <h1>Welcome to ReactJS</h1>;
```

Both examples produce the same output, but JSX is much cleaner and resembles traditional HTML.

Benefits of JSX

- Improves code readability.
- Makes the code easier to understand.
- Reduces the amount of code developers need to write.
- Allows HTML-like syntax inside JavaScript.

How JSX Works?



NU MOA

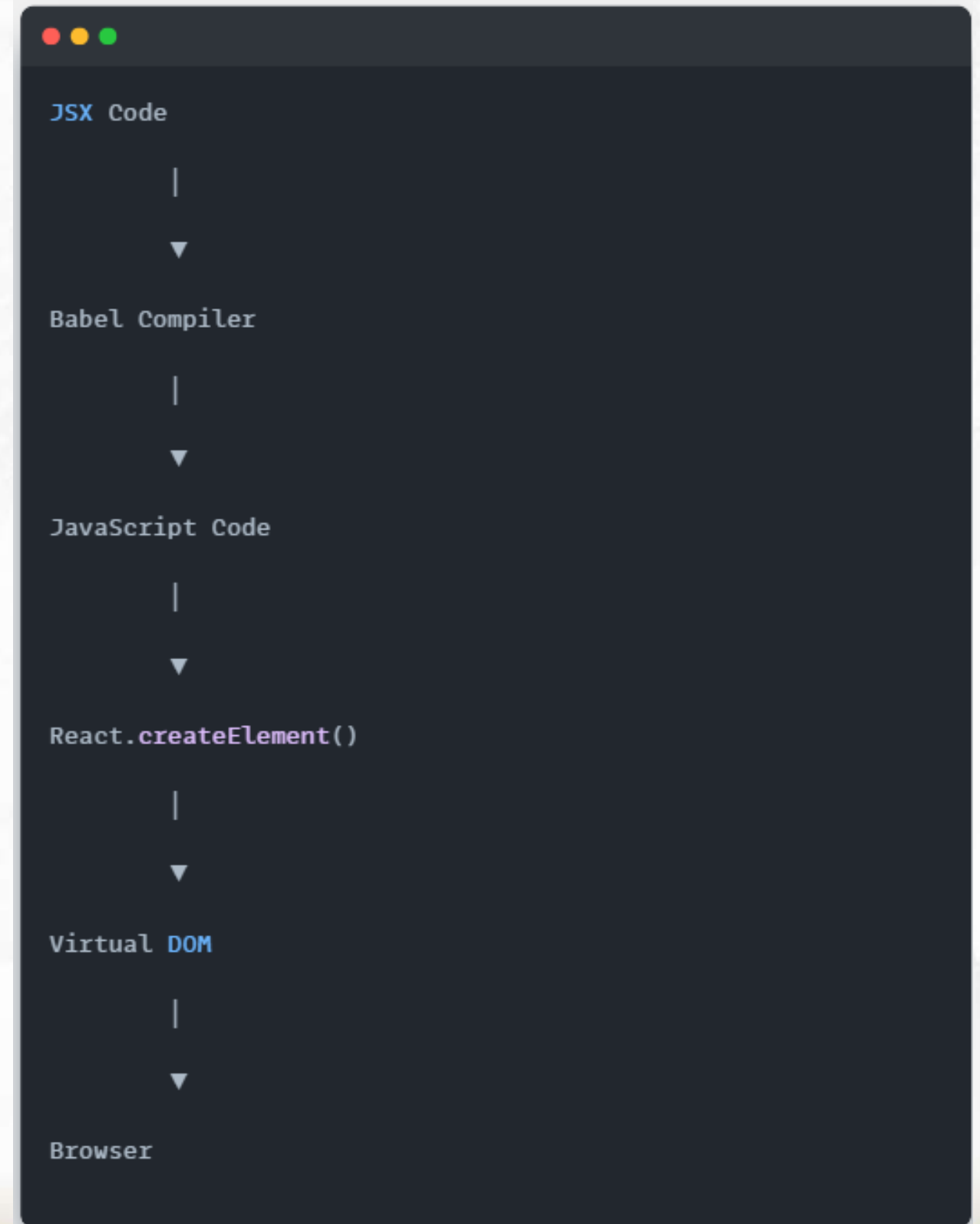


When a developer writes JSX, the browser cannot understand it directly.

Instead, JSX goes through a compilation process before the application runs.

- The developer writes JSX.
- Babel converts JSX into JavaScript.
- React creates Virtual DOM objects.
- React renders the Virtual DOM into the Real DOM.
- The browser displays the webpage.

JSX Compilation Process



Basic JSX Syntax



NU MOA



A JSX element is written similarly to an HTML element.

Example:

```
const title = <h1>Welcome to ReactJS</h1>;
```

React will display:

```
Welcome to ReactJS
```

JSX supports almost all common HTML tags such as:

```
<h1></h1>  
<p></p>  
<div></div>  
<img />  
<button></button>  
<input />
```

This makes JSX familiar to developers who already

To write JSX correctly, developers must follow several important rules.

Rule 1: Return Only One Parent Element

✗ Incorrect

```
return (  
  <h1>Hello</h1>  
  <p>Welcome</p>  
);
```

React cannot return multiple elements without a parent container.

✓ Correct

```
return (  
  <div>  
    <h1>Hello</h1>  
    <p>Welcome</p>  
  </div>  
);
```

A `<div>` acts as the parent element.

To write JSX correctly, developers must follow several important rules.

Example:

```
const student = "Juan";

function App() {
  return (
    <h1>Hello, {student}</h1>
  );
}
```

Output:

Hello, Juan

Developers can insert:

- Variables
- Mathematical operations
- Function calls
- Object properties

To write JSX correctly, developers must follow several important rules.

Rule 2: Close Every Tag

Unlike HTML, JSX requires every tag to be properly closed.

✗ Incorrect

```

```

✓ Correct

```

```

Other Examples:

```
<input />  
<br />  
<hr />
```

Self-closing tags are required in JSX.

JSX uses attributes similar to HTML but with slight differences.

Some HTML attributes have different names in JSX because they conflict with JavaScript keywords.

HTML	JSX
class	className
for	htmlFor
onclick	onClick
tabindex	tabIndex

Advantages of JSX



NU MOA



JSX provides several benefits for React developers.

- ✓ Easy to read and write.
- ✓ Looks similar to HTML.
- ✓ Allows JavaScript and HTML-like syntax in one file.
- ✓ Supports dynamic content.
- ✓ Reduces coding errors.
- ✓ Improves code maintainability.
- ✓ Makes component development easier.

Because of these advantages, JSX has become the standard way of writing React components.

What are React Components?

React Components are the **building blocks of a React application**. A component is an independent and reusable piece of code that represents a specific part of the user interface (UI). Instead of writing an entire webpage in one file, React divides the interface into smaller components that can be reused throughout the application.

Examples of components include a **Header, Navigation Bar, Sidebar, Content Section, and Footer**. By combining these components, developers can build complete and organized web applications.

Why Use React Components?

React Components help developers organize their code into smaller, manageable sections. Since components are reusable, the same component can be used multiple times without rewriting the code.

Benefits

- Reusable code
- Easier maintenance
- Better code organization
- Faster development
- Simplifies debugging

Types of React Components



NU MOA



React provides two types of components:

Functional Components	Class Components
• Created using JavaScript functions. • Simpler and easier to write. • Supports React Hooks. • Recommended for modern React development.	Created using JavaScript classes. Uses the render() method. Commonly used in older React versions.

Functional Component



NU MOA



A Functional Component is a JavaScript function that returns JSX. It is the recommended approach because it is simple, readable, and supports React Hooks.

Example:

```
function Header() {  
  return <h1>My Website</h1>;  
}
```

Class Component



NU MOA



A Class Component is created using JavaScript classes. It was commonly used before React Hooks were introduced.

Example:

```
class Header extends React.Component {  
  render() {  
    return <h1>My Website</h1>;  
  }  
}
```


What is the React Component Life Cycle?

The **React Component Life Cycle** refers to the different stages that a component goes through from the time it is created until it is removed from the webpage.

Just like a human life cycle (birth, growth, and death), a React component also follows a sequence of events during its existence. Understanding these stages helps developers know when a component is created, updated, and removed, allowing them to control how the application behaves.

The Three Main Phases of the Component Life Cycle



NU MOA



Every React component passes through three major phases:

Phase 1: Mounting

Mounting is the first stage of the component life cycle.

During this phase, React creates the component, processes its JSX, and displays it in the browser. This happens only **once**, when the component is first loaded.

Example:

When a user opens a Student Information System, the **Dashboard**, **Header**, and **Navigation Menu** are displayed for the first time. These components are in the **Mounting** phase.

The Three Main Phases of the Component Life Cycle



NU MOA



Every React component passes through three major phases:

Phase 2: Updating

The **Updating** phase occurs whenever the component's data changes.

These changes may come from:

- New Props received from the parent component.
- Changes in the component's State.
- User interactions such as clicking a button or submitting a form.

Instead of refreshing the entire webpage, React updates only the affected component, making the application faster and more efficient.

Example

A student edits their profile information. Only the **Student Profile** component updates, while the rest of the page remains unchanged.

The Three Main Phases of the Component Life Cycle



NU MOA



Every React component passes through three major phases:

Phase 3: Unmounting

Unmounting is the final stage of the component life cycle.

During this phase, React removes the component from the webpage because it is no longer needed.

Removing unused components helps improve application performance and memory management.

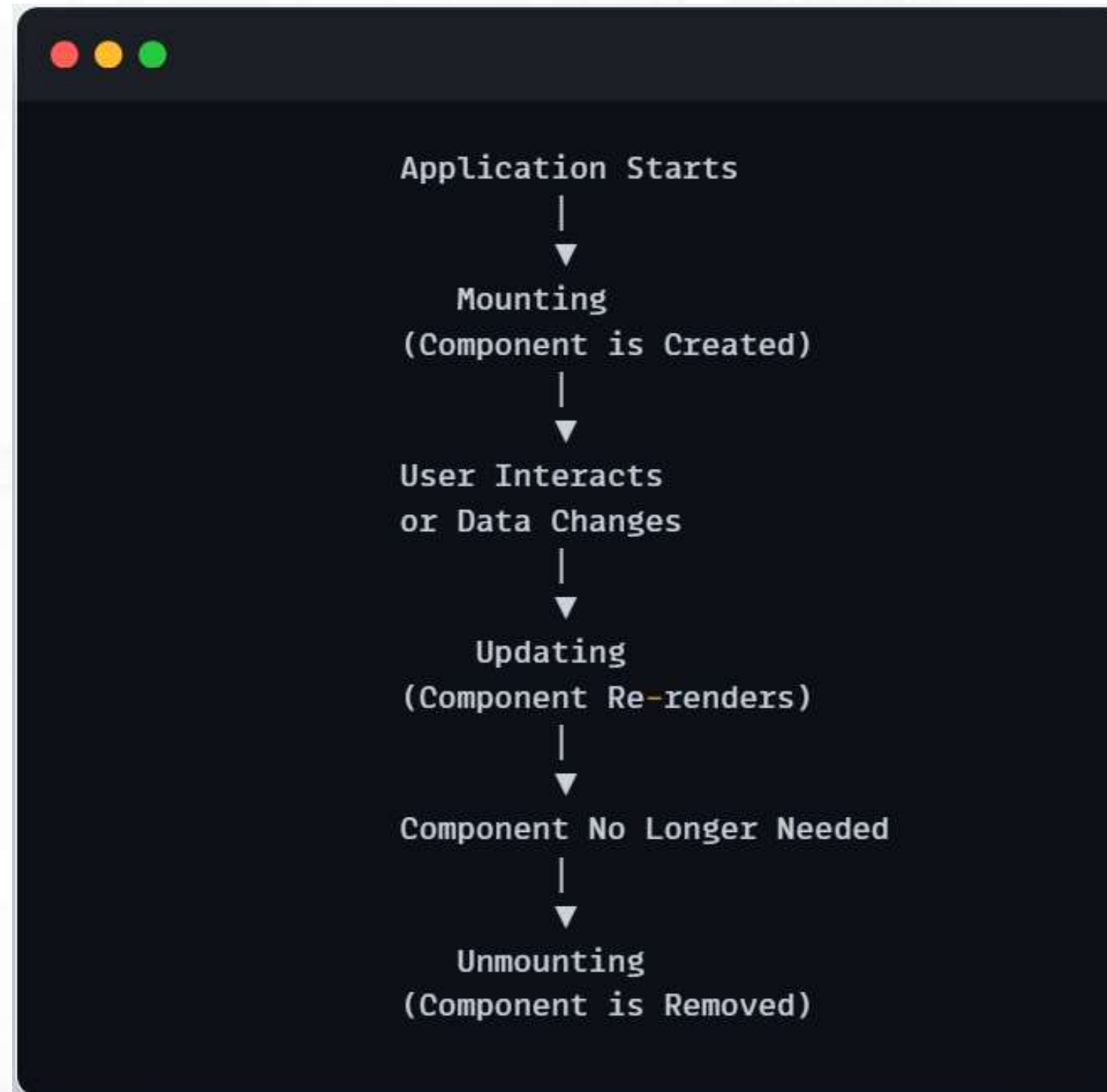
Example

When a user logs out of the system, the **Dashboard** component is removed and replaced by the **Login** component.

React Component Life Cycle Flow



NU MOA



What is React Router?

React Router is a standard library used for navigation in React applications. It allows users to move from one page or component to another **without reloading the entire webpage**.

Unlike traditional websites where each page request loads a new HTML document from the server, React Router updates only the necessary components, providing a faster and smoother user experience.

React Router is commonly used to create **Single Page Applications (SPA)**.

Why Do We Use React Router?

Without React Router, navigating between pages would require the browser to reload the entire webpage every time a user clicks a link.

React Router solves this problem by handling navigation on the client side.

Benefits of React Router

- Provides fast page navigation.
- Prevents full page reloads.
- Improves user experience.
- Makes applications more responsive.
- Organizes multiple pages efficiently.

Example: In an online shopping website, clicking **Home**, **Products**, or **Contact** changes the displayed page instantly without refreshing the browser.

Installing React Router



NU MOA



Before using React Router, it must first be installed in the React project.

Open the terminal and run the following command:

```
npm install react-router-dom
```

After the installation is complete, the package is added to the project's dependencies and can now be imported into React components.

Basic React Router Components

React Router provides several components used for navigation.

Component	Purpose
BrowserRouter	Enables routing for the entire application.
Routes	Groups all route definitions.
Route	Maps a URL path to a React component.
Link	Creates navigation links without reloading the page.

These components work together to control navigation within a React application.

Creating Routes Example



NU MOA



```
import { BrowserRouter, Routes, Route } from "react-router-dom";

function Home() {
  return <h1>Home Page</h1>;
}

function About() {
  return <h1>About Page</h1>;
}

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```




NU MOA



Thank You !!



FOUNDED FOR FILIPINOS, BUILT FOR THE NATION